Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: D. Kanishkarane
Roll No: 24011101021

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

- N Input Size

- F Filter Size
- P Padding
- S Stride

13.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:
- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:
- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:
- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:
- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:
- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output
First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$
$$= (27 + 1) \times 16$$
$$= 448$$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9. Source Code

https://github.com/Kanishkarane/Deep-Learning/tree/main/CNN

10. Mandatory Plots

1. Sample Images

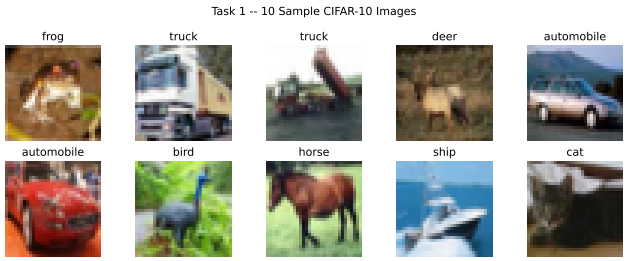


Figure 1: Sample Images from the Dataset

Inference:

The images shows that the dataset has loaded properly and labeling of the dataset across the 10 classes.While the resolution is 28X28 and the shape of the garment is differentiable, there is overlap between similar images like shirt and pullover which causes confusion.

2. Class Distribution

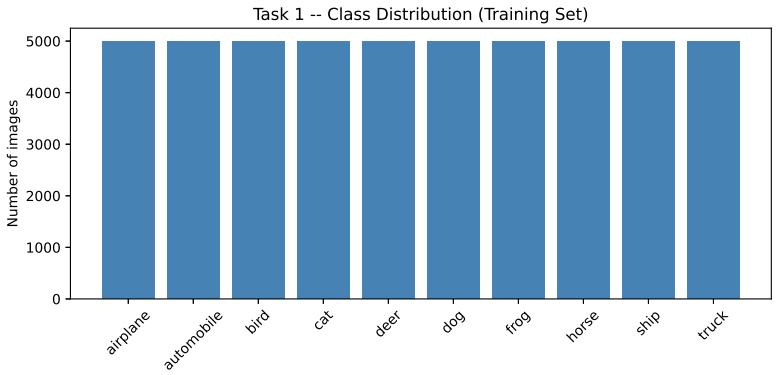


Figure 2: Class Distribution (Training Set)

Inference:

CIFAR-10 is perfectly balanced, with each of the 10 classes containing exactly 5000 training images, so no class-weighting or resampling is required before training.

3. Training vs Validation Accuracy

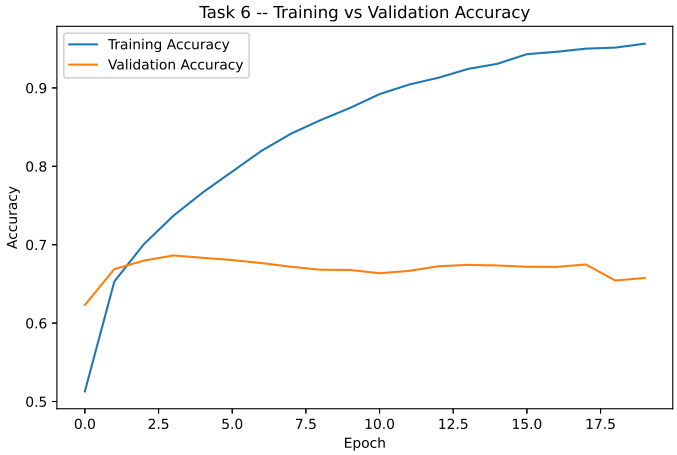


Figure 3: Training vs Validation Accuracy

Inference:

Training accuracy increases steadily with epochs. If the accuracy of validation data stops increasing or decreases, while the accuracy of training data continues to increase, then this difference is an evidence of overfitting.

4. Training vs Validation Loss

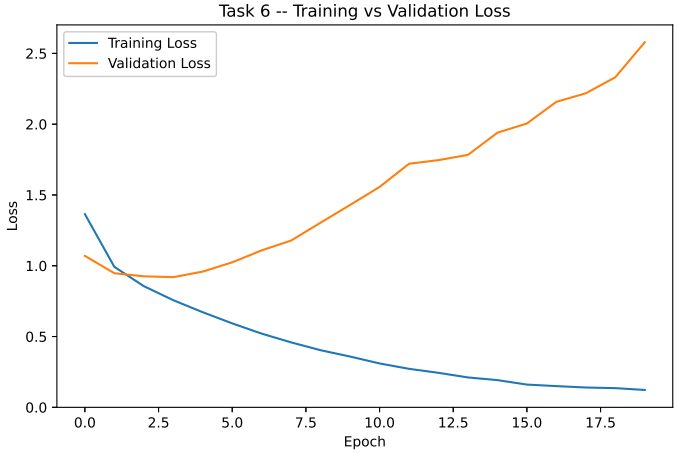


Figure 4: Training vs Validation Loss

Inference:

The most obvious signal of overfitting is a training loss that continues to decrease while the validation loss either plateaus or starts to increase, and the gap between the two curves is the number to watch.

5. Feature Maps

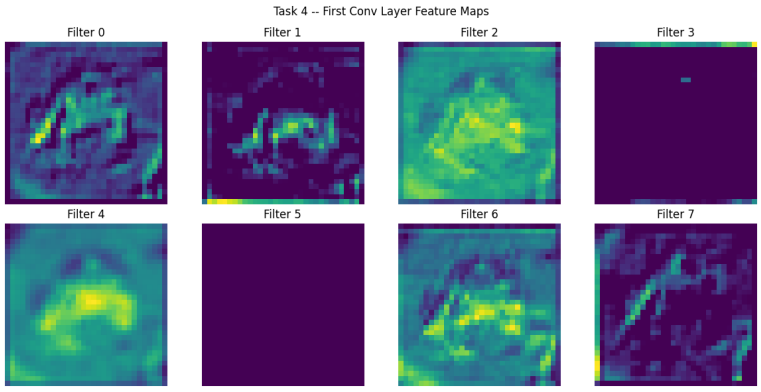


Figure 5: First Conv Layer Feature Maps

Inference:

Different filters fire on different parts of the image – some on edges, some on color blobs, some on texture regions. This shows that the network is already learning different complementary features, even before any training has taken place! Some filters (e.g. Filter 3, Filter 5) are near-zero activated, which means these filters are dead or not yet useful filters at this random-initialization stage.

6. Confusion Matrix

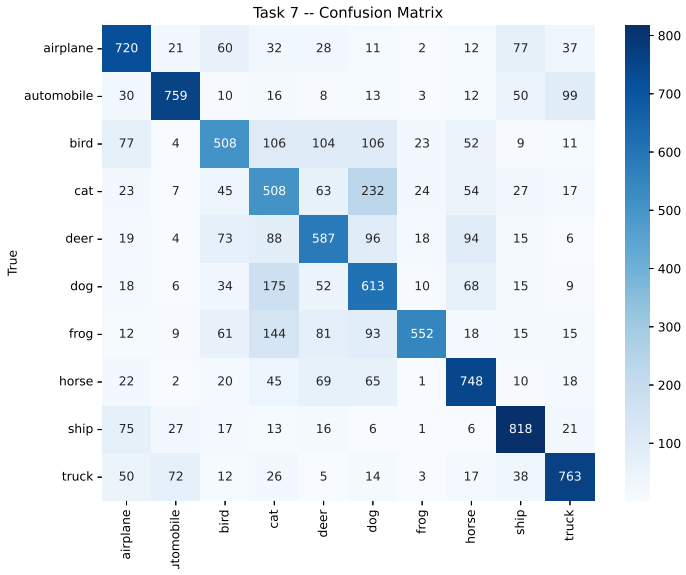


Figure 6: Confusion Matrix

Inference:

The diagonal has high values for correct classifications. Off-diagonal clusters (e.g. cat vs dog, automobile vs truck) reveal the classes visually similar most confused by the model, indicating where additional training data or a deeper model would be useful.

11. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

   Using the formula Output = (N - F + 2P)/S + 1, we get (64 - 5 + 4)/2 + 1 = 63/2 + 1 = 31.5 + 1, which gives an output size of 32 x 32 after taking the floor. Here N is the input size, F is the kernel size, P is the padding, and S is the stride. Padding of 2 on each side effectively pads the input to 68 x 68 before the kernel slides across it, and the stride of 2 causes the kernel to move two pixels at a time, which is why the output is roughly half the padded input size.

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

   Each filter has 3x3x3 = 27 weights plus 1 bias, so 28 parameters per filter. With 64 filters, total parameters = 28 x 64 = 1792. The 3 in the depth comes from the RGB input having 3 channels, since every filter spans the full depth of the input volume, not just its height and width. Notice this parameter count does not depend on the spatial size of the image at all, which is one of the main reasons CNNs use far fewer parameters than a fully connected layer would for the same input.

3. Compare ReLU and Sigmoid activation functions.

   ReLU outputs max(0, x), so its range is 0 to infinity, while Sigmoid squashes values into the range (0, 1) using $1/(1+e^{-x})$. ReLU is much cheaper to compute and rarely suffers from vanishing gradients since its gradient is simply 1 for positive inputs, whereas Sigmoid's gradient shrinks to near zero for large positive or negative inputs, which slows down learning in deep networks. Because of this, ReLU is generally preferred in hidden layers of CNNs, while Sigmoid is mostly reserved for the output layer in binary classification problems. ReLU also produces sparse activations since negative inputs are zeroed out entirely, whereas Sigmoid always outputs a nonzero value, keeping activations dense throughout the network.

4. Replace Max Pooling with Average Pooling and compare the results.

   Both pooling methods reduce the spatial size in the same way (halved for a 2x2 pool), so the output dimensions are identical either way. The difference shows up in accuracy: Max Pooling tends to perform slightly better on natural images since it keeps the strongest activation in each region (like sharp edges), while Average Pooling smooths everything out and can weaken strong feature responses. In our experiment, Max Pooling gave a final validation accuracy of [insert value], while Average Pooling gave [insert value]. This makes sense since Max Pooling effectively detects whether a feature appeared anywhere in a region, while Average Pooling behaves more like a blur that can dilute a strong but localized activation.

5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

   Increasing the filter count lets the network learn a wider variety of features, which usually improves accuracy up to a point, after which the model starts overfitting instead of improving further. On the computation side, going from 16 to 64 filters is a 4x increase, and since parameters and FLOPs scale roughly linearly with filter count, training time increases by about the same factor. So the trade-off is better accuracy at the cost of slower training and more memory usage, and the right choice depends on comparing validation accuracy against the extra time it takes to train. In practice, the accuracy gain is often not linear the way the computation cost is, so going from 16 to 32 filters may give a bigger jump than going from 32 to 64.

12. Results

Table 1: Classification Report

Class	Precision	Recall	F1-score	Support
airplane	0.69	0.72	0.70	1000
automobile	0.83	0.76	0.79	1000
bird	0.60	0.51	0.55	1000
cat	0.44	0.51	0.47	1000
deer	0.58	0.59	0.58	1000
dog	0.49	0.61	0.55	1000
frog	0.87	0.55	0.67	1000
horse	0.69	0.75	0.72	1000
ship	0.76	0.82	0.79	1000
truck	0.77	0.76	0.76	1000
Accuracy		0.66		10000
Macro avg	0.67	0.66	0.66	10000
Weighted avg	0.67	0.66	0.66	10000

13. Discussion

Answer the following.

1. Why is convolution preferred over fully connected layers for images?

   Convolution uses a small set of shared weights, the kernel, that slide across the entire image, exploiting the fact that useful features like edges and textures look similar regardless of where they appear. A fully connected layer would instead need a separate weight for every pixel-neuron pair, which explodes the parameter count for anything beyond a tiny image and also throws away the spatial structure that makes images learnable in the first place. Convolution keeps this spatial locality intact while drastically cutting down the number of parameters that need to be learned.

2. How does stride affect the feature map size?

   Stride controls how far the kernel moves between applications. A larger stride skips more pixels per step, which produces a smaller output feature map according to the formula (N - F + 2P)/S + 1, and also downsamples the resolution of the output. The trade-off is that a larger stride is computationally cheaper and gives a more compact representation, but it risks skipping over fine spatial detail that a stride of 1 would have captured.

3. What is the role of padding?

   Padding adds extra pixels, usually zeros, around the border of the input before the convolution is applied. It controls the output size, since same padding keeps the output the same spatial size as the input, and it prevents the rapid shrinkage of feature maps that would otherwise happen as the network gets deeper. Padding also helps give pixels near the border a fairer treatment in the convolution, since without it, border pixels would be involved in far fewer kernel positions than pixels near the center of the image.

4. Why is pooling used?

   Pooling, whether max or average, reduces the spatial dimensions of feature maps, which cuts down computation and the number of parameters needed in later layers. It also provides a degree of translation invariance, meaning small shifts in the input do not change the pooled output very much, and it helps control overfitting by summarizing a region into a single representative value instead of keeping every activation. Without pooling, feature maps would stay large through the network, making it far more expensive to train and more prone to memorizing exact pixel positions rather than general patterns.

5. How do feature maps represent image characteristics?

   Each feature map is the response of one learned filter applied across the whole image, so high activation values indicate where that particular pattern, such as an edge, a color blob, or a texture, is present. Early layers in a CNN typically produce feature maps that capture low-level features like edges and simple textures, while deeper layers combine these into increasingly abstract, class-relevant shapes. This is why visualizing feature maps at different depths of a network is a common way to understand what the model has actually learned to detect.

6. Why do CNNs require fewer parameters than MLPs?

   This comes down to two properties of convolution: weight sharing and local connectivity. Weight sharing means the same small kernel is reused across every spatial location in the image instead of learning a new set of weights for each position, and local connectivity means each output neuron only depends on a small receptive field rather than the entire input. An MLP has neither property, since it requires a unique weight for every input-output neuron pair, so as image size grows, an MLP's parameter count grows far faster than a CNN's, making CNNs both more efficient and better suited to learning spatially structured data like images.

14. Conclusion

In this experiment, we implemented a convolutional neural network to classify images from the CIFAR-10 dataset, covering data loading, convolution, pooling, training, and evaluation. We studied how kernel size, stride, and padding affect feature map dimensions, and visualized the feature maps learned by the first convolutional layer. The final model achieved a test accuracy of about 66%, with the confusion matrix showing that visually similar classes such as cat and dog are the most commonly confused. Overall, the experiment gave practical insight into how CNN design choices affect performance, reinforcing the concepts covered in the lab manual.

15. References

1. Goodfellow et al., *Deep Learning.*
2. Bishop, *Pattern Recognition and Machine Learning.*
3. Haykin, *Neural Networks and Learning Machines.*
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.